



**UNIVERSIDADE FEDERAL DE RORAIMA**



ALONSO FERNANDES CAVALCANTE

RAUL SILVA PAIVA

RUAN FELIPE LAURIANO RAMOS

## **INSTRUÇÕES PARA A EXECUÇÃO DO TRABALHO PRÁTICO I**

Estrutura De Dados I - 2025.2

Boa Vista, RR  
2025

Alonso Fernandes Cavalcante, Raul Silva Paiva, Ruan Felipe Lauriano Ramos

## SUMÁRIO

|                                                     |          |
|-----------------------------------------------------|----------|
| <b>SUMÁRIO</b>                                      | <b>1</b> |
| <b>1. INTRODUÇÃO</b>                                | <b>2</b> |
| <b>2. DESENVOLVIMENTO</b>                           | <b>2</b> |
| <b>2.1. Estrutura e Conformidade com Requisitos</b> | <b>2</b> |
| <b>2.2. Processo de Indexação</b>                   | <b>2</b> |
| <b>2.3. Instruções de Uso</b>                       | <b>3</b> |
| <b>2.4. Operações do Menu</b>                       | <b>3</b> |
| <b>3. CONCLUSÃO</b>                                 | <b>4</b> |

## 1. INTRODUÇÃO

O presente trabalho descreve a implementação de um sistema de indexação de texto baseado na estrutura de dados de índice invertido. O objetivo principal deste projeto é processar arquivos de texto para criar um dicionário dinâmico que mapeia palavras às suas respectivas localizações (linhas) e frequências no arquivo, simulando o funcionamento do núcleo de um motor de busca. A solução foi desenvolvida na linguagem C, priorizando o uso de memória dinâmica e Tipos Abstratos de Dados (TADs) para garantir eficiência e modularidade.

## 2. DESENVOLVIMENTO

### 2.1. Estrutura e Conformidade com Requisitos

O software foi arquitetado para atender integralmente às exigências de alocação dinâmica e proibição de estruturas estáticas globais. A base do sistema é composta por duas estruturas principais (structs):

- **Dicionário (Palavra):** Uma lista encadeada simples ordenada alfabeticamente. Cada nó armazena uma string (`char*`) alocada dinamicamente e um ponteiro para a sub-lista de ocorrências.
- **Lista de Ocorrências (Ocorrencia):** Uma lista encadeada subordinada a cada palavra. Cada nó armazena o número da linha e a quantidade de vezes que a palavra apareceu naquela linha específica.

Essa estrutura hierárquica permite que o sistema gerencie textos de tamanho arbitrário, limitado apenas pela memória disponível, cumprindo o requisito de uso de TADs e estruturas dinâmicas.

### 2.2. Processo de Indexação

O algoritmo de indexação opera através de uma varredura sequencial do arquivo de entrada:

1. **Leitura e Tokenização:** O arquivo é lido linha a linha. Cada linha é fragmentada em *tokens* (palavras) utilizando delimitadores padrão (espaço, pontuação, quebras de linha).

2. **Busca e Inserção Ordenada:** Para cada token identificado, o sistema percorre a lista principal (**Dicionário**).
  - Se a palavra não existe, um novo nó é alocado e inserido na posição correta para manter a ordem alfabética.
  - Se a palavra já existe, o algoritmo acessa sua sub-lista de ocorrências.
3. **Registro de Ocorrência:** Na sub-lista, verifica-se se a linha atual já possui registro.
  - Caso positivo, o contador de frequência daquela linha é incrementado.
  - Caso negativo, um novo nó de ocorrência é criado para a linha atual.

### 2.3. Instruções de Uso

O programa foi desenvolvido para ser compilado via GCC e executado em terminal.

```
PS C:\Users\chest\Documents\2. Áreas\Acadêmico\2025.2\Estrutura de Dados I\TrabalhoPráticoI> gcc main.c indice.c -o main.exe
PS C:\Users\chest\Documents\2. Áreas\Acadêmico\2025.2\Estrutura de Dados I\TrabalhoPráticoI> ./main.exe
Nome do arquivo (ex: entrada.txt):
```

- **Compilação:** Deve-se compilar os módulos **main.c** e **indice.c** conjuntamente.
  - Comando: **gcc main.c indice.c -o indexador**
- **Execução:**
  - Comando: **./indexador** (Linux/Mac) ou **indexador.exe** (Windows).
- **Entrada de Dados:** Ao iniciar, o programa solicita o nome do arquivo de texto (ex: **entrada.txt**). O arquivo deve estar no mesmo diretório do executável.

### 2.4. Operações do Menu

Após o processamento do arquivo, o sistema oferece um menu interativo com as seguintes funcionalidades obrigatórias e extras:

- 1. **Imprimir Índice Invertido:** Exibe todas as palavras do dicionário, seguidas pela lista de pares (**Linha, Frequência**), conforme o formato de saída especificada.
- 2. **Buscar Palavra:** Permite ao usuário digitar um termo. O sistema percorre a lista e, se encontrar, exibe detalhadamente as linhas e frequências associadas.
- 3. **Remover Palavra:** Localiza e remove uma palavra do dicionário. Esta operação libera a memória de todos os nós de ocorrência associados antes

de liberar o nó da palavra e a string, garantindo que não haja vazamento de memória (*memory leak*).

- **4. Palavra Mais Frequente:** Realiza um cálculo estatístico percorrendo todo o índice para somar as ocorrências de cada palavra, identificando e exibindo a que possui a maior frequência global no texto.

### **3. CONCLUSÃO**

A implementação realizada cumpriu todos os objetivos propostos para a disciplina de Estrutura de Dados. A utilização de listas encadeadas aninhadas demonstrou-se uma estratégia eficaz para a representação de índices invertidos, permitindo inserções ordenadas e buscas eficientes sem o desperdício de memória típico de vetores estáticos. O programa resultante é capaz de indexar, buscar e manipular dados textuais de forma robusta, atendendo aos critérios de pontuação máxima estabelecidos.